# NirmanIQ Internship — Day 13 Handbook

| MODULE | CORE TECHNOLOGY | AUTHOR |
| --- | --- | --- |
| Backend Foundations | PostgreSQL 16 & TypeORM in NestJS | Penumuru Madhu Sudhan Reddy |

**Executive Summary:** Day 13 transitions our enterprise NestJS backend from volatile in-memory JavaScript arrays to a persistent, highly scalable relational database (PostgreSQL 16) using TypeORM. We implement UUID primary keys, snake_case column mapping, hierarchical entity relations (Project → Tower → Floor), safe migration lifecycles, and high-performance QueryBuilder multi-table aggregations.

## 1. Why PostgreSQL in Construction Intelligence Systems

In construction platforms like NirmanIQ, relational databases are critical due to strict relational hierarchies and data integrity requirements:

- **ACID Transactions:** Updating floor completion, deducting inventory, and logging safety inspections must either all succeed together or all roll back together.
- **Foreign Key Constraints:** Enforces on disk that a floor can never reference a non-existent tower, and deleting a project safely cascades down to child assets.
- **Relational Performance:** Complex analytical queries across hundreds of floors and towers execute in single-digit milliseconds using multi-table JOINs and GROUP BY aggregations.

## 2. NirmanIQ Database Conventions

| Convention | Implementation | Rationale |
| --- | --- | --- |
| **UUID Primary Keys** | `@PrimaryGeneratedColumn('uuid')` | Eliminates ID enumeration (IDOR attacks) and enables offline client apps to generate unique IDs before syncing. |
| **snake_case in SQL** | `@Column({ name: 'created_at' })` | PostgreSQL defaults to case-insensitive lowercasing. snake_case preserves word separation in SQL. |
| **Never SELECT *** | `select: ['id', 'name', 'status']` | Prevents loading heavy text and audit fields, lowering memory usage and network transfer times. |

## 3. Entity Models & Relational Architecture

### ProjectEntity (Parent)

```
@Entity('projects')
export class ProjectEntity {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ length: 100 })
  name: string;

  @Column({ type: 'enum', enum: ProjectStatus })
  status: ProjectStatus;

  @OneToMany(() => TowerEntity, t => t.project, { cascade: true })
  towers: TowerEntity[];
}
```

### TowerEntity (Child & Parent)

```
@Entity('towers')
export class TowerEntity {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ name: 'risk_level' })
  riskLevel: number;

  @ManyToOne(() => ProjectEntity, p => p.towers, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'project_id' })
  project: ProjectEntity;

  @OneToMany(() => FloorEntity, f => f.tower)
  floors: FloorEntity[];
}
```

## 4. Migrations & Schema Evolution

In production, `synchronize: false` is strictly enforced. Schema changes are managed via version-controlled TypeORM migrations:

```
export class AddIsArchivedToProjects1728980001000 implements MigrationInterface {
  public async up(queryRunner: QueryRunner): Promise {
    await queryRunner.query(`ALTER TABLE "projects" ADD COLUMN "is_archived" boolean NOT NULL DEFAULT false;`);
  }
  public async down(queryRunner: QueryRunner): Promise {
    await queryRunner.query(`ALTER TABLE "projects" DROP COLUMN "is_archived";`);
  }
}
```

## 5. QueryBuilder Analytics vs. Repository

While standard CRUD uses `repository.find()`, cross-table metrics use `QueryBuilder` with `getRawMany()` to eliminate N+1 queries:

```
return await this.projectRepository
  .createQueryBuilder('p')
  .leftJoin('p.towers', 't')
  .leftJoin('t.floors', 'f')
  .select('p.id', 'projectId')
  .addSelect('p.name', 'projectName')
  .addSelect('COUNT(DISTINCT t.id)', 'towerCount')
  .addSelect('ROUND(COALESCE(AVG(f.completion_percentage), 0), 2)', 'avgCompletion')
  .addSelect('COALESCE(MAX(t.risk_level), 0)', 'maxRiskLevel')
  .where('p.is_archived = :isArchived', { isArchived: false })
```

```
    .groupBy('p.id')
    .getRawMany();
```